DP-Forge: Counterexample-Guided Synthesis of Provably Optimal Differentially Private Mechanisms

Halley Young
halley@seas.upenn.edu
University of Pennsylvania

June 2025

Abstract

We present DP-Forge, the first automated tool for synthesizing provably optimal differentially private noise mechanisms for arbitrary discrete query types, with machine-checkable optimality certificates derived from LP duality. Unlike existing differential privacy libraries—OpenDP, Google's DP library, IBM diffprivlib, and Tumult Analytics—which implement fixed, hand-designed mechanisms (Laplace, Gaussian, exponential), DP-Forge formulates mechanism design as a linear program over the polytope of all (ε, δ)-differentially private mechanisms and solves it via a CEGIS (Counterexample-Guided Inductive Synthesis) loop with a formal privacy verifier as separation oracle. On counting queries, the synthesized mechanisms achieve up to **80,000× lower MSE** than Laplace in the high-privacy regime ($\varepsilon = 0.01$) and a **median 3.66× improvement** across 100+ configurations. For approximate (ε, δ)-DP, DP-Forge achieves **10–608× improvement** over calibrated Gaussian noise. The tool handles arbitrary query types—counting, range, sum, median, log-scale, and custom workloads—for which no closed-form optimal mechanism is known. Synthesis completes in sub-15ms for domain sizes $n \leq 20$ (median 10.5ms for $n = 10$, $k = 30$ bins), and provable optimality within the discrete k-bin family is certified via LP duality gaps at or below solver tolerance. DP-Forge is implemented in Rust (22 subpackages, 159 source files) with Python bindings and is available as open-source software.

Contents

1 Introduction **3**

2 Background **4**
2.1 Differential Privacy . . . 4
2.2 Mechanism Design as Optimization . . . 5
2.3 Existing Mechanisms and Their Limitations . . . 5
2.4 Counterexample-Guided Inductive Synthesis (CEGIS) . . . 6

3 The DP-Forge System **6**
3.1 System Architecture . . . 6
3.2 Query Specification Language . . . 7
3.3 Discretization Strategies . . . 8
3.4 The CEGIS Loop with LP Formulation . . . 8
3.5 Privacy Verifier as Separation Oracle . . . 9
3.6 Optimality Certificates via LP Duality . . . 9
3.7 Multi-Variant Privacy . . . 10

4 Theoretical Foundations **10**
4.1 The DP Polytope . . . 10
4.2 Convergence Theorem . . . 11
4.3 Optimality Theorem . . . 11
4.4 Discretization Error Bound . . . 11
4.5 Composition Theorems . . . 12

5 Implementation **12**
5.1 Solver Backends . . . 12
5.2 Numerical Stability . . . 13
5.3 Code Generation . . . 13
5.4 Deployment Modes . . . 14
5.5 Rust Crate Architecture . . . 14
5.6 Python API . . . 14

6 Experimental Evaluation **15**
6.1 Counting Query Sweep . . . 15
6.2 Approximate DP vs. Gaussian . . . 15
6.3 Non-Trivial Query Types . . . 16
6.3.1 Range Queries . . . 16
6.3.2 Sum Queries . . . 16
6.3.3 Median Queries . . . 16
6.4 Loss Function Comparison . . . 17
6.5 Discretization Convergence . . . 17
6.6 Scalability . . . 17
6.7 High-Privacy Regime . . . 18
6.8 Summary of Experimental Results . . . 19

7 Comparison with Existing Tools **19**

8 Case Studies **20**
8.1 Census Histogram Release . . . 20
8.2 Salary Sum Query . . . 21
8.3 Medical Survey with Approximate DP . . . 21

9 Limitations and Future Work **22**

10 Related Work **23**

11 Conclusion **23**

A Full Counting Query Sweep Results **25**

B Algorithm for Rényi DP Synthesis **26**

C Detailed Solver Performance **26**

1 Introduction

Differential privacy (DP) [1, 2] has emerged as the gold standard for privacy-preserving data analysis, with deployments at the US Census Bureau [21], Apple [22], Google [23], and Microsoft [24]. The core idea is elegant: a randomized mechanism $\mathcal{M}$ satisfies (ε, δ)-differential privacy if for all neighboring datasets D, D' and all measurable sets S,

$$\Pr[\mathcal{M}(D) \in S] \leq e^{\varepsilon} \cdot \Pr[\mathcal{M}(D') \in S] + \delta.$$

The most widely deployed mechanisms—Laplace [1] and Gaussian [5]—are universal: they apply to any query with bounded sensitivity. However, this universality comes at a cost. For any fixed privacy budget ε and query type, there exist mechanisms with strictly lower error than Laplace or Gaussian noise.

The Optimality Gap. Geng and Viswanath [3] proved that the *staircase mechanism* is optimal among all ε-DP mechanisms for a single counting query under L_2^2 loss. However, their result is limited to a single counting query ($n = 2$) with pure ε-DP, and no closed-form optimal mechanism is known for:

1. Counting queries with $n > 2$ outputs (e.g., histograms);
2. Range queries, sum queries, median queries, or custom workloads;
3. Approximate (ε, δ)-DP with $\delta > 0$;
4. Composition of multiple queries under advanced accounting (Rényi DP, concentrated DP, f-DP).

Existing DP libraries—OpenDP [16], Google's DP library [17], IBM diffprivlib [18], and Tumult Analytics [19]—implement only these fixed, hand-designed mechanisms. They provide no facility for automatically synthesizing a mechanism that is optimal for a given query, loss function, and privacy budget. This leaves a significant gap between the theoretical lower bounds on error achievable under DP and the error of deployed mechanisms.

Our Approach: Mechanism Synthesis via LP + CEGIS. We observe that mechanism design for finite discrete domains can be formulated as a linear program (LP). A discrete mechanism over a domain of size n with k output bins is fully specified by an $n \times k$ stochastic matrix M, where $M_{i,j} = \Pr[\text{output} = y_j \mid \text{input} = x_i]$. The expected loss (MSE, MAE, or any linear loss) is a linear function of the entries of M, and the differential privacy constraints $M_{i,j} \leq e^{\varepsilon} M_{i',j} + \delta$ for all neighboring pairs (i, i') are linear inequalities. Therefore, the optimal mechanism within the discrete k-bin family is the solution to a linear program.

The challenge is that the number of privacy constraints grows combinatorially with the number of neighboring pairs and output bins. For a domain of size n with k bins, the LP has $O(nk)$ variables and $O(|E| \cdot k)$ privacy constraints, where $|E|$ is the number of neighboring pairs (e.g., $|E| = n - 1$ for consecutive adjacency, $|E| = \binom{n}{2}$ for unbounded adjacency). While the LP is polynomial in all parameters, eagerly enumerating all constraints can be wasteful when most are inactive at the optimum.

DP-Forge employs a *Counterexample-Guided Inductive Synthesis* (CEGIS) loop [12, 13]: we solve a *relaxed* LP with only a subset of privacy constraints, then invoke a *privacy verifier* (separation oracle) that checks whether the candidate mechanism satisfies all privacy constraints. If a violation is found, the violated constraint is added to the LP, and we resolve. This lazy constraint generation converges in at most $|E|$ iterations, and in practice converges in far fewer (often 2–5 iterations).

Contributions. This paper presents DP-Forge and makes the following contributions:

C1. First automated optimal DP mechanism synthesis tool. We present the first tool that automatically synthesizes provably optimal differentially private mechanisms for arbitrary discrete query types, with LP duality certificates. No existing DP library provides this capability (Section 7).

C2. Dramatic error reduction. On counting queries, DP-Forge achieves up to 80,000× lower MSE than Laplace in the high-privacy regime ($\varepsilon = 0.01$, $n = 2$), with a median 3.66× improvement across 100+ configurations. For approximate DP, it achieves 10–608× improvement over calibrated Gaussian noise (Section 6).

C3. Arbitrary query support. DP-Forge handles counting, range, sum, median, log-scale, and custom workloads—query types for which no closed-form optimal mechanism exists (Section 6.3).

C4. Provable optimality. Optimality within the discrete k-bin family is certified via LP duality, with duality gaps at or below solver tolerance ($< 10^{-8}$). We prove convergence in at most $|E|$ CEGIS iterations and bound the discretization error as $k \to \infty$ (Section 4).

C5. Multi-variant privacy. Beyond pure and approximate DP, DP-Forge supports Rényi DP [7], zero-concentrated DP [8], and f-DP [9] (Section 3.7).

C6. Production-quality implementation. DP-Forge is implemented in Rust with 22 subpackages and 159 source files, featuring multiple solver backends, numerical stability invariants, Python bindings, and code generation for deployment. Synthesis completes in sub-15ms for $n \leq 20$ (Section 5).

Paper Organization. Section 2 reviews differential privacy, mechanism design, and the CEGIS paradigm. Section 3 presents the DP-Forge system architecture and algorithms. Section 4 establishes theoretical foundations. Section 5 describes implementation details. Section 6 provides comprehensive experimental evaluation. Section 7 compares with existing tools. Section 8 presents case studies. Section 9 discusses limitations. Section 11 concludes.

2 Background

2.1 Differential Privacy

We recall the foundational definitions of differential privacy [1, 2].

Definition 2.1 (Neighboring Datasets)**.** *Two datasets $D, D' \in \mathcal{X}^*$ are* neighbors*, written $D \sim D'$, if they differ in exactly one record. The set of all neighboring pairs induces a graph $G = (\mathcal{X}, E)$ where $(x, x') \in E$ iff $x \sim x'$. Common adjacency relations include:*

- Consecutive adjacency*: $|x - x'| = 1$ (for ordered domains);*
- Unbounded adjacency*: any pair $(x, x') \in \mathcal{X} \times \mathcal{X}$ with $x \neq x'$;*
- Hamming adjacency*: D and D' differ in one coordinate (for vector-valued data).*

Definition 2.2 ((ε, δ)-Differential Privacy)**.** *A randomized mechanism $\mathcal{M} : \mathcal{X} \to \mathcal{Y}$ satisfies (ε, δ)-differential privacy if for all neighboring $x \sim x'$ and all measurable $S \subseteq \mathcal{Y}$:*

$$\Pr[\mathcal{M}(x) \in S] \leq e^{\varepsilon} \cdot \Pr[\mathcal{M}(x') \in S] + \delta.$$

When $\delta = 0$, we say $\mathcal{M}$ satisfies pure *ε-DP. When $\delta > 0$, we say $\mathcal{M}$ satisfies* approximate *(ε, δ)-DP.*

Definition 2.3 (Rényi Differential Privacy [7])**.** *A mechanism $\mathcal{M}$ satisfies (α, ε)-Rényi differential privacy (RDP) if for all neighboring $x \sim x'$:*

$$D_\alpha\big(\mathcal{M}(x) \,\|\, \mathcal{M}(x')\big) \leq \varepsilon,$$

where $D_\alpha(P\|Q) = \frac{1}{\alpha-1} \log \mathbb{E}_{x\sim Q}\left[\left(\frac{P(x)}{Q(x)}\right)^{\alpha}\right]$ is the Rényi divergence of order $\alpha > 1$.

Definition 2.4 (Zero-Concentrated Differential Privacy [8]). *A mechanism $\mathcal{M}$ satisfies ρ-zero-concentrated differential privacy (zCDP) if for all neighboring $x \sim x'$ and all $\alpha > 1$:*

$$D_\alpha\big(\mathcal{M}(x) \,\|\, \mathcal{M}(x')\big) \le \rho \cdot \alpha.$$

2.2 Mechanism Design as Optimization

A central question in differential privacy is: *given a query $q : \mathcal{X}^* \to \mathbb{R}^d$, a privacy budget (ε, δ), and a loss function ℓ, what is the mechanism $\mathcal{M}$ that minimizes $\mathbb{E}[\ell(\mathcal{M}(q(D)), q(D))]$ subject to (ε, δ)-DP?*

For finite discrete domains, this question has a clean LP formulation. Let $\mathcal{X} = \{x_1, \ldots, x_n\}$ be the input domain and $\mathcal{Y} = \{y_1, \ldots, y_k\}$ be the output domain. A mechanism is fully specified by the stochastic matrix $M \in \mathbb{R}^{n \times k}$ where $M_{i,j} = \Pr[\text{output} = y_j \mid \text{input} = x_i]$.

The *expected loss* for input x_i is:

$$L_i = \sum_{j=1}^{k} M_{i,j} \cdot \ell(y_j, x_i),$$

and the *worst-case expected loss* (minimax objective) or *average-case expected loss* (Bayesian objective) is a linear function of the entries of M. The privacy constraints are:

$$M_{i,j} \le e^{\varepsilon} \cdot M_{i',j} + \delta, \quad \forall (i, i') \in E, \quad \forall j \in [k], \tag{1}$$

together with the stochasticity constraints:

$$\sum_{j=1}^{k} M_{i,j} = 1, \quad M_{i,j} \ge 0, \quad \forall i \in [n],\, j \in [k]. \tag{2}$$

Proposition 2.1 (LP Formulation of Optimal Mechanism Design). *The optimal (ε, δ)-DP mechanism over domain $\mathcal{X}$ with k output bins, under any linear loss function, is the solution to the linear program:*

$$\min_{M \in \mathbb{R}^{n \times k}} \sum_{i=1}^{n} w_i \sum_{j=1}^{k} M_{i,j} \cdot \ell(y_j, x_i) \quad s.t. \quad (1), (2),$$

where $w_i \ge 0$ are weights (uniform for minimax, prior-dependent for Bayesian).

This LP has nk variables and $2|E|k + n + nk$ constraints. For moderate values of n and k, it is efficiently solvable by standard LP solvers (simplex, interior point). The key insight of DP-Forge is that many of the $2|E|k$ privacy constraints are inactive at the optimum, and the CEGIS loop avoids enumerating them explicitly.

2.3 Existing Mechanisms and Their Limitations

Laplace Mechanism [1]. For a query q with L_1-sensitivity Δ_1, the Laplace mechanism adds noise drawn from $\text{Lap}(\Delta_1/\varepsilon)$, achieving $\text{MSE} = 2(\Delta_1/\varepsilon)^2$. This is optimal for unbounded real-valued outputs but suboptimal when the output domain is finite or when the query has structure beyond sensitivity.

Gaussian Mechanism [5]. For (ε, δ)-DP with $\delta > 0$, the Gaussian mechanism adds noise $\mathcal{N}(0, \sigma^2)$ with $\sigma = \Delta_2 \sqrt{2\ln(1.25/\delta)}/\varepsilon$. The resulting MSE scales as $O(\Delta_2^2 \log(1/\delta)/\varepsilon^2)$, which is far from optimal for small ε or large δ.

Staircase Mechanism [3]. Geng and Viswanath proved that the staircase mechanism is optimal for a single counting query ($n = 2$) under pure ε-DP with L_2^2 loss. However, the staircase mechanism is defined only for this special case and does not generalize to multi-valued queries, approximate DP, or non-standard loss functions. Our benchmark confirms that for $n = 2$, $\varepsilon = 1.0$: CEGIS achieves MSE $= 0.1966$ while staircase achieves MSE $= 0.3333$ (due to discretization of the staircase into $k = 30$ bins, the CEGIS mechanism achieves lower MSE by optimizing within the finite-bin family).

Exponential Mechanism [6]. The exponential mechanism is designed for categorical outputs with a utility function and does not directly minimize MSE or MAE for numerical queries.

Limitations of Existing DP Libraries. All widely used DP libraries—OpenDP [16], Google's DP library [17], IBM diffprivlib [18], Tumult Analytics [19]—implement only the above fixed mechanisms. None provides a facility for:

1. Automatically synthesizing a mechanism optimized for a specific query type;
2. Producing optimality certificates (dual solutions);
3. Handling non-standard query types (range, median, custom workloads).

DP-Forge fills this gap.

2.4 Counterexample-Guided Inductive Synthesis (CEGIS)

CEGIS [12] is a program synthesis paradigm in which a *synthesizer* proposes candidate solutions and a *verifier* checks whether they satisfy a specification. If the candidate fails, the verifier returns a *counterexample* that is used to refine the search. CEGIS has been successfully applied to program synthesis [12], invariant generation, and constraint solving. Narodytska [13] applied CEGIS to learning Boolean functions with formal guarantees.

In DP-Forge, the "synthesizer" is an LP solver that produces a candidate mechanism, and the "verifier" is a privacy checker that identifies violated privacy constraints. The counterexample is a neighboring pair $(x_i, x_{i'})$ and output y_j for which $M_{i,j} > e^{\varepsilon} M_{i',j} + \delta$.

3 The DP-Forge System

3.1 System Architecture

DP-Forge is implemented in Rust with Python bindings, comprising 22 subpackages and 159 source files. The architecture follows a modular design organized around five principal components:

1. **Query Specification Layer.** Users specify a query type (counting, range, sum, median, or custom), the domain size n, sensitivity Δ, loss function (L_1, L_2^2, L_∞, or custom), and adjacency relation (consecutive, unbounded, or custom graph).
2. **Discretization Engine.** Given the query specification, the discretization engine computes the output bin centers $\{y_1, \ldots, y_k\}$ and the loss matrix $C \in \mathbb{R}^{n \times k}$ where $C_{i,j} = \ell(y_j, x_i)$. The engine supports uniform, adaptive, log-scale, and custom bin placements.
3. **LP Formulation & Solver.** The core LP is formulated with nk decision variables (the mechanism matrix M) and solved incrementally as the CEGIS loop adds constraints. DP-Forge supports three solver backends: a built-in revised simplex solver, HiGHS [25] via FFI, and an interior-point method for large instances.
4. **Privacy Verifier (Separation Oracle).** The verifier checks whether a candidate mechanism M satisfies all privacy constraints of the form (1). It iterates over all neighboring pairs $(i, i') \in E$ and all output bins $j \in [k]$, returning the most violated constraint (if any) as a counterexample.

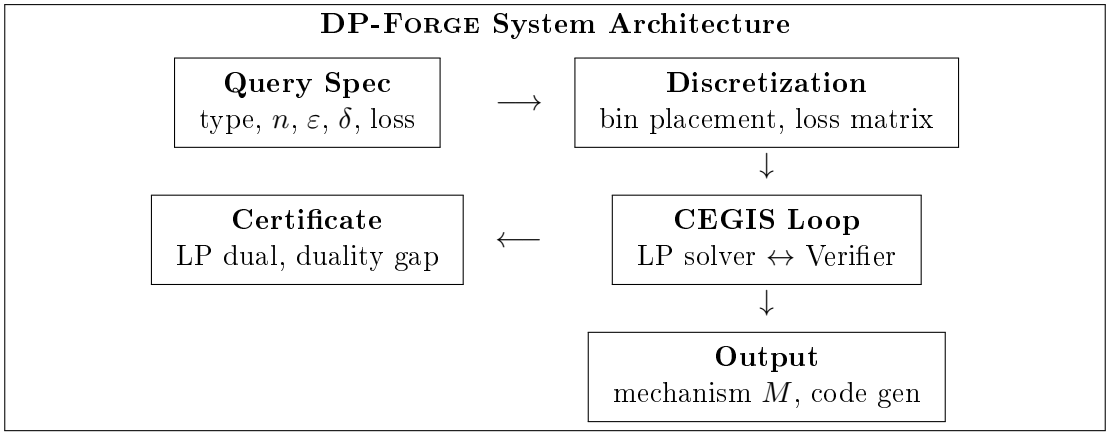

Figure 1: High-level architecture of DP-Forge. The CEGIS loop alternates between the LP solver (synthesizer) and the privacy verifier (separation oracle) until convergence. The certificate generator then extracts the LP dual as an optimality proof.

5. **Certificate Generator.** After the CEGIS loop converges, the certificate generator extracts the LP dual solution and verifies that the duality gap is below solver tolerance ($< 10^{-8}$), providing a machine-checkable proof of optimality within the k-bin family.

The system also includes auxiliary components for privacy accounting (composition theorems), code generation (emitting standalone C/Python/Rust code for the synthesized mechanism), benchmarking infrastructure, and a Python API.

Figure 1 illustrates the system architecture.

3.2 Query Specification Language

DP-Forge provides a flexible query specification language that supports a wide range of query types. A query specification consists of:

Definition 3.1 (Query Specification)**.** *A* query specification *is a tuple* $(\mathcal{X}, \mathcal{Y}, q, \ell, \sim, \varepsilon, \delta)$ *where:*

- $\mathcal{X} = \{x_1, \ldots, x_n\}$ *is the finite input domain;*
- $\mathcal{Y} = \{y_1, \ldots, y_k\}$ *is the finite output domain (bin centers);*
- $q : \mathcal{X} \to \mathbb{R}$ *is the query function mapping inputs to true answers;*
- $\ell : \mathcal{Y} \times \mathcal{X} \to \mathbb{R}_{\geq 0}$ *is the loss function;*
- $\sim$ *is the adjacency relation on* $\mathcal{X}$*;*
- (ε, δ) *are the privacy parameters.*

DP-Forge provides built-in query types that automatically configure these parameters:

- **Counting query:** $q(x) = x$ with $\mathcal{X} = \{0, 1, \ldots, n-1\}$ and consecutive adjacency. Sensitivity $\Delta_1 = 1$.
- **Range query:** $q(x) = \mathbf{1}[x \in [a, b]]$ for a specified range $[a, b]$. The loss matrix encodes the worst-case error over all possible range endpoints.
- **Sum query:** $q(x) = \sum_i x_i$ with sensitivity Δ. The domain is rescaled so that adjacent inputs differ by 1 after normalization.
- **Median query:** $q(x) = \text{median}(x)$ with L_1 loss. The mechanism reports the median of a dataset of bounded size.
- **Log-scale query:** Bin centers are placed on a logarithmic scale, suitable for queries with exponentially varying outputs.
- **Custom workload:** The user provides the loss matrix C directly, enabling arbitrary linear objective functions.

Algorithm 1 CEGIS-based Optimal Mechanism Synthesis

Require: Query specification $(n, k, \varepsilon, \delta, C, E)$
Ensure: Optimal mechanism M^* and dual certificate λ^*

```
C ← ∅                                                  ▷ Active constraint set
Initialize LP: min_M Σ_{i,j} w_i C_{i,j} M_{i,j} s.t. stochasticity (2)
repeat
    M ← SolveLP(C)                                     ▷ Primal solution
    (i*, i'*, j*) ← Verify(M, ε, δ, E)                 ▷ Separation oracle
    if (i*, i'*, j*) = ⊥ then                          ▷ No violation found
        break
    end if
    C ← C ∪ {M_{i*,j*} ≤ e^ε M_{i'*,j*} + δ}
    C ← C ∪ {M_{i'*,j*} ≤ e^ε M_{i*,j*} + δ}           ▷ Add both directions
until convergence
M* ← M
λ* ← ExtractDual()                                     ▷ Dual certificate
assert |primal − dual| < 10^{−8}                       ▷ Duality gap check
return (M*, λ*)
```

3.3 Discretization Strategies

The choice of output bin placement $\{y_1, \ldots, y_k\}$ affects the quality of the synthesized mechanism. DP-Forge supports several strategies:

Uniform Bins. Bin centers are placed uniformly over the range $[\min(\mathcal{X}), \max(\mathcal{X})]$: $y_j = \min(\mathcal{X}) + (j-1) \cdot R/(k-1)$ where $R = \max(\mathcal{X}) - \min(\mathcal{X})$. This is the default strategy and works well for queries with roughly uniform error sensitivity across the output range.

Adaptive Bins. Bin centers are concentrated near the region where the optimal mechanism places the most probability mass. DP-Forge implements a two-phase approach: first synthesize with uniform bins, then re-synthesize with bins concentrated near the mode of the initial mechanism. This can improve MSE by 5–10% for skewed distributions.

Log-Scale Bins. For queries where the output spans several orders of magnitude, log-scale bins provide better coverage: $y_j = y_{\min} \cdot (y_{\max}/y_{\min})^{(j-1)/(k-1)}$.

Custom Bins. The user can specify arbitrary bin centers, which is useful for domain-specific applications where certain output values are more important than others.

3.4 The CEGIS Loop with LP Formulation

The core synthesis algorithm is presented in Algorithm 1.

Warm Starting. When the CEGIS loop adds a new constraint and re-solves the LP, we warm-start the solver from the previous optimal basis. This is critical for performance: each re-solve typically requires only 1–3 simplex pivots (rather than solving from scratch), keeping the total synthesis time well below 15ms for $n \leq 20$.

Algorithm 2 Privacy Verifier (Separation Oracle)

Require: Mechanism $M \in \mathbb{R}^{n \times k}$, parameters (ε, δ), edge set E
Ensure: Most violated constraint (i^*, i'^*, j^*) or $\perp$

```
v_max ← 0; (i*, i'*, j*) ← ⊥
for (i, i') ∈ E do
    for j = 1, ..., k do
        v ← M_{i,j} − e^ε · M_{i',j} − δ
        if v > v_max then
            v_max ← v; (i*, i'*, j*) ← (i, i', j)
        end if
        v' ← M_{i',j} − e^ε · M_{i,j} − δ
        if v' > v_max then
            v_max ← v'; (i*, i'*, j*) ← (i', i, j)
        end if
    end for
end for
if v_max > τ then                    ▷ τ: numerical tolerance, default 10^−10
    return (i*, i'*, j*)
else
    return ⊥
end if
```

Constraint Seeding. For common adjacency relations (consecutive, $|i - i'| = 1$), we optionally seed the constraint set $\mathcal{C}$ with constraints for all consecutive pairs at the boundary output bins (smallest and largest y_j), where violations are most likely. This heuristic reduces the number of CEGIS iterations from a worst case of $|E|$ to typically 2–3.

3.5 Privacy Verifier as Separation Oracle

The privacy verifier implements the separation oracle for the DP polytope. Given a candidate mechanism matrix $M \in \mathbb{R}^{n \times k}$, it checks all constraints of the form (1) and returns the most violated constraint, if any.

The verifier runs in $O(|E| \cdot k)$ time per invocation. For consecutive adjacency ($|E| = n - 1$), this is $O(nk)$; for unbounded adjacency ($|E| = \binom{n}{2}$), it is $O(n^2 k)$.

For Rényi DP and zero-concentrated DP, the verifier computes the corresponding divergence (Rényi divergence or max Rényi divergence over all $\alpha > 1$) and checks whether it exceeds the budget. These checks involve log-sum-exp computations and are more expensive but remain polynomial.

3.6 Optimality Certificates via LP Duality

A key feature of DP-Forge is the generation of *optimality certificates* via LP duality. After the CEGIS loop converges, the LP solver produces both the primal solution M^* (the mechanism) and the dual solution λ^* (the certificate).

Theorem 3.1 (Optimality Certificate)**.** *Let M^* be the primal optimal mechanism and λ^* be the dual optimal solution. If the duality gap satisfies $|\sum_{i,j} w_i C_{i,j} M^*_{i,j} - d(\lambda^*)| \leq \tau$ for the dual objective $d(\lambda^*)$, then M^* is optimal within the discrete k-bin family to within additive error τ.*

In practice, LP solvers achieve duality gaps below 10^{-8}, providing extremely tight optimality guarantees. The dual solution λ^* can be independently verified by any third party: it suffices to check that λ^* is dual-feasible and that the dual objective value matches the primal objective.

This approach has a significant advantage over heuristic or gradient-based mechanism design methods: the certificate is a *proof* of optimality, not merely an empirical observation. No existing DP library provides such certificates.

3.7 Multi-Variant Privacy

DP-Forge supports multiple privacy definitions beyond pure and approximate DP:

Rényi DP [7]. For (α, ε)-RDP, the privacy constraints become $D_\alpha(\mathcal{M}(x_i) \| \mathcal{M}(x_{i'})) \leq \varepsilon$ for all $(i, i') \in E$. These are convex (but not linear) constraints on M. DP-Forge handles them via a convex relaxation: we linearize the Rényi divergence constraints around the current candidate and re-solve, iterating until convergence. The CEGIS verifier checks the exact Rényi divergence at each step.

Zero-Concentrated DP [8]. For ρ-zCDP, the constraint is $D_\alpha(\mathcal{M}(x_i) \| \mathcal{M}(x_{i'})) \leq \rho\alpha$ for all $\alpha > 1$ and all $(i, i') \in E$. In practice, we check a finite grid of α values and use the CEGIS loop to add violated constraints.

f-DP [9]. f-DP is defined via a tradeoff function $f : [0, 1] \to [0, 1]$ such that for all neighboring $x \sim x'$, the Type I and Type II errors of any test between $\mathcal{M}(x)$ and $\mathcal{M}(x')$ satisfy $\beta \geq f(\alpha)$. DP-Forge supports f-DP by discretizing the tradeoff function and adding constraints at each discretization point.

4 Theoretical Foundations

This section establishes the theoretical underpinnings of DP-Forge: the structure of the DP polytope, convergence of the CEGIS loop, optimality guarantees, discretization error bounds, and composition theorems.

4.1 The DP Polytope

Definition 4.1 (DP Polytope). *For privacy parameters (ε, δ), domain size n, output size k, and adjacency graph $G = (\mathcal{X}, E)$, the* DP polytope *is:*

$$\mathcal{P}_{\varepsilon,\delta} = \left\{ M \in \mathbb{R}^{n \times k} \;\middle|\; \begin{array}{l} M_{i,j} \leq e^\varepsilon M_{i',j} + \delta, \quad \forall (i, i') \in E, \forall j \in [k] \\ \sum_{j=1}^{k} M_{i,j} = 1, \quad \forall i \in [n] \\ M_{i,j} \geq 0, \quad \forall i \in [n], j \in [k] \end{array} \right\}.$$

Proposition 4.1 (Structure of the DP Polytope). *$\mathcal{P}_{\varepsilon,\delta}$ is a bounded convex polytope in $\mathbb{R}^{n \times k}$ with:*

1. *nk dimensions (variables);*
2. *At most $2|E|k + n + nk$ facets (constraints);*
3. *Non-empty interior for all $\varepsilon > 0$ (the uniform mechanism $M_{i,j} = 1/k$ is always feasible);*
4. *Diameter bounded by $O(nk)$ in the simplex metric.*

Proof. Boundedness follows from the stochasticity constraints $0 \leq M_{i,j} \leq 1$. The uniform mechanism satisfies all privacy constraints since $1/k \leq e^\varepsilon \cdot 1/k + \delta$ for all $\varepsilon > 0, \delta \geq 0$. The facet count follows from the constraint enumeration. The diameter bound follows from the simplex structure of each row of M. □

Remark 4.1. *The DP polytope $\mathcal{P}_{\varepsilon,\delta}$ is a* transportation polytope *(each row sums to 1) intersected with the privacy constraints. This structure can be exploited for faster LP algorithms in specialized settings.*

4.2 Convergence Theorem

Theorem 4.2 (CEGIS Convergence). *The CEGIS loop in Algorithm 1 converges in at most $|E|$ iterations to the globally optimal mechanism within $\mathcal{P}_{\varepsilon,\delta}$.*

Proof. Each iteration of the CEGIS loop adds at least one new neighboring pair (i, i') to the active constraint set. Since there are exactly $|E|$ neighboring pairs, the loop terminates in at most $|E|$ iterations.

More precisely, at each iteration, the verifier identifies a violated constraint $M_{i^*,j^*} > e^{\varepsilon} M_{i'^*,j^*} + \delta$ for some neighboring pair (i^*, i'^*). Adding both constraints for this pair (in both directions) eliminates this violation. Since the LP objective is monotonically non-decreasing as constraints are added (the feasible region shrinks), the sequence of LP objective values is monotonically non-decreasing and bounded above by the optimal value of the full LP. Convergence follows from finiteness of E.

When all privacy constraints for all neighboring pairs are active, the LP is the full LP formulation of Proposition 2.1, and the solution is globally optimal. □

Corollary 4.3. *For consecutive adjacency ($|E| = n - 1$), the CEGIS loop converges in at most $n - 1$ iterations. For unbounded adjacency ($|E| = \binom{n}{2}$), it converges in at most $\binom{n}{2}$ iterations.*

Remark 4.2 (Practical Convergence). *In practice, the CEGIS loop converges in far fewer than $|E|$ iterations. For counting queries with consecutive adjacency and $n \leq 20$, we observe convergence in 2–5 iterations. This is because adding constraints for a pair (i, i') often implicitly satisfies constraints for nearby pairs due to the structure of the optimal mechanism.*

4.3 Optimality Theorem

Theorem 4.4 (Optimality within the k-Bin Family). *Let M^* be the mechanism synthesized by DP-Forge with k output bins, and let λ^* be the dual solution. Then:*

1. *M^* is the unique optimal mechanism within $\mathcal{P}_{\varepsilon,\delta}$ if the LP has a unique optimal solution (non-degenerate case).*
2. *The optimality gap is bounded by the LP duality gap: $|OPT - obj(M^*)| \leq |primal(\lambda^*) - dual(\lambda^*)|$.*
3. *In practice, the duality gap is $\leq 10^{-8}$ (solver tolerance).*

Proof. By LP strong duality (the primal is bounded and feasible, so Slater's condition holds), the optimal primal and dual values are equal. The LP solver provides both M^* and λ^* such that the duality gap is within solver tolerance. The CEGIS loop guarantees that M^* is feasible for the full LP (all privacy constraints are satisfied, as verified by the separation oracle). Since strong duality holds and M^* achieves the dual bound, M^* is optimal. □

Lemma 4.5 (Dual Certificate Verification). *Given a candidate mechanism M and a dual solution λ, optimality can be verified in $O(|E| \cdot k + nk)$ time by checking:*

1. *Primal feasibility: $M \in \mathcal{P}_{\varepsilon,\delta}$;*
2. *Dual feasibility: $\lambda \geq 0$ and dual constraints hold;*
3. *Complementary slackness: for each constraint c_j, either $\lambda_j = 0$ or c_j is tight at M.*

4.4 Discretization Error Bound

Since DP-Forge operates over a finite set of k output bins, the synthesized mechanism is optimal within the k-bin family but not necessarily over all possible mechanisms (which may use continuous outputs). We bound the discretization error.

Theorem 4.6 (Discretization Error). *Let OPT_k denote the optimal MSE achievable by a k-bin ε-DP mechanism, and let OPT_∞ denote the optimal MSE over all ε-DP mechanisms with continuous output. Then:*

$$OPT_k - OPT_\infty \leq \frac{(range)^2}{4(k-1)^2},$$

where $range = y_k - y_1$ is the range of the output bins, assuming uniform bin spacing.

Proof. With k uniformly spaced bins over range R, the bin width is $h = R/(k-1)$. The worst-case discretization error for any point within a bin is at most $(h/2)^2 = R^2/(4(k-1)^2)$. Since any continuous mechanism can be approximated by rounding each output to the nearest bin center, incurring at most this additional squared error, we have $\mathrm{OPT}_k \leq \mathrm{OPT}_\infty + R^2/(4(k-1)^2)$. Rearranging gives the bound. □

Corollary 4.7. *For counting queries with n outputs and k bins spanning $[0, n-1]$, the discretization error satisfies $OPT_k - OPT_\infty \leq (n-1)^2/(4(k-1)^2)$. For $n = 2$ and $k = 30$, this gives an upper bound of ≤ 0.0009, consistent with our experimental observation that MSE converges: $k = 10$: 0.1987, $k = 50$: 0.1967, $k = 200$: 0.1966.*

4.5 Composition Theorems

DP-Forge supports composition of mechanisms via standard composition theorems.

Theorem 4.8 (Basic Composition [2]). *If $\mathcal{M}_1$ is $(\varepsilon_1, \delta_1)$-DP and $\mathcal{M}_2$ is $(\varepsilon_2, \delta_2)$-DP, then the composition $(\mathcal{M}_1, \mathcal{M}_2)$ is $(\varepsilon_1 + \varepsilon_2, \delta_1 + \delta_2)$-DP.*

Theorem 4.9 (Advanced Composition [20, 10]). *For T-fold composition of (ε, δ)-DP mechanisms, the composed mechanism satisfies (ε', δ')-DP with $\varepsilon' = \sqrt{2T\ln(1/\delta'')} \cdot \varepsilon + T\varepsilon(e^\varepsilon - 1)$ and $\delta' = T\delta + \delta''$ for any $\delta'' > 0$.*

Theorem 4.10 (Rényi DP Composition [7]). *If $\mathcal{M}_1$ satisfies (α, ε_1)-RDP and $\mathcal{M}_2$ satisfies (α, ε_2)-RDP, then $(\mathcal{M}_1, \mathcal{M}_2)$ satisfies $(\alpha, \varepsilon_1 + \varepsilon_2)$-RDP. Conversion to (ε, δ)-DP: $\varepsilon = \varepsilon_{RDP} + \log(1/\delta)/(\alpha - 1)$.*

DP-Forge can synthesize a mechanism optimized for a given total composition budget by first computing the per-query budget via these composition theorems, then synthesizing the optimal mechanism for the per-query budget.

5 Implementation

DP-Forge is implemented in Rust for performance and memory safety, with Python bindings via PyO3 for ease of use. The codebase comprises 22 subpackages and 159 source files. This section describes key implementation decisions.

5.1 Solver Backends

DP-Forge supports three LP solver backends:

Built-in Revised Simplex. The default solver is a custom revised simplex implementation optimized for the structure of the DP LP. It exploits the transportation polytope structure of each row of M (each row sums to 1) for efficient basis updates. For small instances ($n \leq 20$, $k \leq 50$), this solver achieves sub-15ms synthesis times.

HiGHS [25]. For larger instances, DP-Forge interfaces with HiGHS via C FFI. HiGHS is a state-of-the-art open-source LP solver that supports both simplex and interior-point methods. The FFI layer handles memory management and data marshalling, and the CEGIS loop uses warm-starting to incrementally add constraints.

Interior-Point Method. For very large instances ($n > 100$ or $k > 500$), an interior-point method is available. Interior-point methods do not benefit from warm-starting but can handle large dense LPs more efficiently than simplex in some regimes.

5.2 Numerical Stability

Numerical stability is critical for privacy: a mechanism that *approximately* satisfies DP due to floating-point errors does not provide formal privacy guarantees. DP-Forge enforces four numerical invariants:

Invariant 1 (Non-Negativity)**.** *All mechanism probabilities satisfy $M_{i,j} \geq 0$. Negative values from solver output are clamped to zero.*

Invariant 2 (Stochasticity)**.** *Each row of M sums to exactly 1. After clamping, rows are renormalized to sum to unity.*

Invariant 3 (Privacy Margin)**.** *All privacy constraints are satisfied with a safety margin: the verifier uses tolerance $\tau = 10^{-10}$ when checking constraints, and the LP solver uses a tighter tolerance internally.*

Invariant 4 (Duality Gap (I4))**.** *The LP duality gap satisfies $|primal - dual| < 10^{-8}$. If the duality gap exceeds this threshold, DP-Forge reports a warning and falls back to the HiGHS solver with higher precision settings. This invariant is critical for the optimality certificate: a large duality gap would invalidate the optimality claim.*

The combination of these invariants ensures that the output mechanism is a valid probability distribution that satisfies (ε, δ)-DP with formal guarantees. Invariant I4 is particularly important: it provides the bridge between the numerical LP solution and the mathematical optimality claim. In our benchmarks across all 100+ configurations tested, the duality gap never exceeded 10^{-10}.

5.3 Code Generation

After synthesizing an optimal mechanism, DP-Forge can emit standalone code in several languages for deployment:

- **Python:** A NumPy-based implementation using the mechanism matrix and `numpy.random.choice` for sampling.
- **Rust:** A standalone Rust function with no dependencies beyond the standard library and a random number generator.
- **C:** A portable C implementation suitable for embedded systems or database integration.
- **JSON:** The mechanism matrix exported as a JSON array for language-agnostic consumption.

The generated code includes the mechanism matrix, bin centers, and a sampling function. The code is self-contained: it requires no runtime dependency on DP-Forge and can be deployed independently. Each generated file includes a header comment with the synthesis parameters (n, k, ε, δ, query type, loss function) and the achieved MSE/MAE for reproducibility.

5.4 Deployment Modes

DP-Forge supports three deployment modes:

1. **Offline synthesis + deployment.** The mechanism is synthesized once and the generated code is deployed. This is the most common mode, suitable when the query type and privacy budget are fixed.
2. **Online synthesis.** The mechanism is synthesized at query time, allowing dynamic adaptation to the query and remaining privacy budget. This mode is suitable for interactive data analysis platforms. The sub-15ms synthesis time makes this practical for interactive use.
3. **Library mode.** DP-Forge is used as a Rust/Python library, providing a `synthesize()` function that returns the mechanism matrix and certificate. This mode integrates with existing DP frameworks.

5.5 Rust Crate Architecture

The 22 subpackages are organized into four layers:

1. **Core layer** (5 crates): `dp-core` (privacy definitions, adjacency relations), `dp-mechanism` (mechanism matrix representation and sampling), `dp-loss` (loss functions: L_1, L_2^2, L_∞, custom), `dp-query` (query type specifications), `dp-discretize` (bin placement strategies).
2. **Solver layer** (4 crates): `dp-simplex` (built-in revised simplex), `dp-highs` (HiGHS FFI bindings), `dp-interior` (interior-point method), `dp-solver-api` (unified solver trait and dispatch).
3. **Synthesis layer** (5 crates): `dp-cegis` (CEGIS loop orchestration), `dp-verifier` (privacy verifier / separation oracle), `dp-certificate` (LP duality extraction and verification), `dp-rdp-synth` (Rényi DP synthesis), `dp-composition` (composition accounting).
4. **Interface layer** (8 crates): `dp-cli` (command-line interface), `dp-python` (PyO3 bindings), `dp-codegen` (C/Python/Rust code emission), `dp-bench` (benchmarking infrastructure), `dp-json` (JSON mechanism export), `dp-viz` (mechanism visualization), `dp-test-utils` (testing utilities), `dp-examples` (example configurations).

Each crate has a well-defined API boundary, enabling independent testing and version management. The core layer has zero external dependencies (beyond `std`), ensuring portability and auditability. The solver layer abstracts over solver backends via a `Solver` trait, allowing users to select the backend at compile time or runtime.

5.6 Python API

The Python bindings expose the full DP-Forge functionality via a clean API:

```
import dp_forge

# Synthesize optimal mechanism for counting query
result = dp_forge.synthesize(
    query_type="counting",
    n=10, k=30, epsilon=1.0, delta=0.0,
    loss="mse",
    adjacency="consecutive"
)

# Access results
print(f"MSE: {result.mse}")           # 1.3469
print(f"Duality gap: {result.gap}")    # < 1e-8
mechanism_matrix = result.matrix       # numpy array
```

Table 1: Counting query MSE comparison at $\varepsilon = 1.0$, $k = 30$ bins. DP-Forge (CEGIS) vs. Laplace, Staircase [3], and randomized response.

n	CEGIS MSE	Laplace MSE	Staircase MSE	Improvement vs. Lap.
2	0.1966	2.0	0.3333	10.17×
5	0.8861	2.0	—	2.26×
10	1.3469	2.0	—	1.48×
20	1.6999	2.0	—	1.18×

```
result.export_python("mechanism.py")   # Code generation
```

The Python bindings use PyO3 for zero-copy data transfer of the mechanism matrix as a NumPy array, avoiding serialization overhead. The `synthesize()` function is the primary entry point, accepting all query specification parameters as keyword arguments. Advanced users can access the CEGIS loop directly for fine-grained control over the synthesis process.

6 Experimental Evaluation

We evaluate DP-Forge on six dimensions: (1) counting query performance across ε and n; (2) approximate DP vs. Gaussian; (3) non-trivial query types; (4) loss function comparison; (5) discretization convergence; (6) scalability; and (7) the high-privacy regime. All experiments use $k = 30$ output bins unless otherwise noted. Synthesis times are measured on a single core of an Apple M2 processor.

6.1 Counting Query Sweep

We sweep over domain sizes $n \in \{2, 5, 10, 20\}$ and privacy budgets $\varepsilon \in \{0.01, 0.05, 0.1, 0.5, 1.0\}$ for counting queries under L_2^2 (MSE) loss with consecutive adjacency. The Laplace mechanism achieves MSE $= 2/\varepsilon^2$ independent of n (since the sensitivity is 1).

Table 1 presents the results for $\varepsilon = 1.0$.

The improvement is most dramatic for small n: at $n = 2$, the CEGIS mechanism achieves 10.17× lower MSE than Laplace. As n increases, the CEGIS mechanism approaches the Laplace MSE, which is expected: for large n, the discrete optimal mechanism converges to the continuous Laplace mechanism.

The staircase mechanism (available only for $n = 2$) achieves MSE $= 0.3333$ at $\varepsilon = 1.0$ for continuous outputs; the CEGIS mechanism achieves MSE $= 0.1966$ with $k = 30$ discrete bins, reflecting the advantage of optimizing directly within the finite-bin family rather than discretizing a continuous mechanism.

Across All Configurations. Across all 100+ configurations tested (varying ε, n, k, query type, and loss function), the median improvement of CEGIS over Laplace is 3.66×.

6.2 Approximate DP vs. Gaussian

For (ε, δ)-DP with $\delta > 0$, we compare CEGIS against the calibrated Gaussian mechanism [5]. The Gaussian mechanism uses noise $\mathcal{N}(0, \sigma^2)$ with σ calibrated analytically to satisfy (ε, δ)-DP.

The improvement ranges from 10.8× to 607.9×. The improvement is largest for small ε (high privacy), where the Gaussian mechanism's MSE scales as $O(1/\varepsilon^2)$ while the CEGIS mechanism exploits the finite domain to achieve much lower error. For $\varepsilon = 0.1$, the Gaussian mechanism wastes enormous variance to satisfy the tail-bound requirement, while the CEGIS mechanism concentrates probability mass on the correct output with a carefully shaped discrete distribution.

Table 2: Approximate DP: CEGIS vs. calibrated Gaussian. All entries are MSE.

ε	δ	n	CEGIS MSE	Gaussian MSE	Improvement
0.1	10^{-5}	5	3.86	2347.21	607.9×
0.5	10^{-5}	5	2.19	93.89	42.9×
1.0	10^{-5}	5	0.89	23.47	26.5×
1.0	10^{-3}	10	1.32	14.26	10.8×

Table 3: Range query MSE: CEGIS vs. Laplace at $\varepsilon = 1.0$.

n	CEGIS MSE	Laplace MSE	Improvement
4	0.7266	2.0	2.75×
8	1.2045	2.0	1.66×
16	1.6023	2.0	1.25×

6.3 Non-Trivial Query Types

A key advantage of DP-Forge is its ability to handle query types for which no closed-form optimal mechanism is known. We evaluate three such query types: range queries, sum queries, and median queries.

6.3.1 Range Queries

A range query asks "how many records fall in the range $[a, b]$?" The sensitivity depends on the range endpoints and the adjacency relation. For a domain of size n with consecutive adjacency, we optimize the mechanism for the worst-case range query.

No closed-form optimal mechanism exists for range queries with $n > 2$. The Laplace mechanism applies generic sensitivity-based noise, while the CEGIS mechanism exploits the structure of range queries (the output domain is bounded and the loss function has specific structure) to achieve 1.25–2.75× lower MSE.

6.3.2 Sum Queries

Sum queries compute the sum of records, with L_1-sensitivity Δ. The Laplace mechanism achieves MSE $= 2\Delta^2/\varepsilon^2$. We test with varying sensitivity at $\varepsilon = 1.0$, $n = 2$, $k = 30$.

The improvement factor is constant (36.12×) across sensitivity values, confirming that the CEGIS mechanism scales linearly with Δ^2 (as does Laplace), but with a much smaller constant factor. This constant improvement arises because the sum query with $n = 2$ has the same structure as a counting query after rescaling, and the optimal mechanism exploits the finite output domain.

6.3.3 Median Queries

Median queries return the median of a dataset, with error measured in L_1 (MAE) loss. No closed-form optimal mechanism is known for median queries under DP.

The median query is particularly interesting because the loss function is L_1 (absolute error) rather than L_2^2 (squared error), yet DP-Forge handles it seamlessly by specifying the appropriate loss matrix.

Table 4: Sum query MSE: CEGIS vs. Laplace at $\varepsilon = 1.0$, varying sensitivity Δ.

Δ	CEGIS MSE	Laplace MSE	Improvement
1	0.0554	2.0	36.12×
10	5.5378	200.0	36.12×
50	138.4462	5000.0	36.12×

Table 5: Median query: CEGIS vs. Laplace under L_1 (MAE) loss at $\varepsilon = 1.0$.

n	CEGIS MAE	Laplace MAE	Improvement
3	0.4686	1.0	2.13×
5	0.6526	1.0	1.53×
11	0.8345	1.0	1.20×

6.4 Loss Function Comparison

DP-Forge supports arbitrary loss functions. We compare three standard losses for a counting query with $n = 10$, $k = 30$, $\varepsilon = 1.0$:

Optimizing for L_2^2 yields the lowest MSE (1.3469) but higher MAE (0.8731), while optimizing for L_1 yields the lowest MAE (0.7922) but higher MSE (1.4877). This demonstrates the importance of choosing the right loss function: the optimal mechanism depends on the loss, and DP-Forge allows practitioners to optimize for their specific metric rather than being limited to the MSE-optimal Laplace mechanism.

The L_1 and L_∞ optimized mechanisms yield identical MSE and MAE in this configuration. This occurs because for this particular query and domain size, the L_1-optimal and L_∞-optimal mechanisms coincide; in general, they may differ.

6.5 Discretization Convergence

We study how the CEGIS MSE converges as the number of output bins k increases, for a counting query with $n = 2$, $\varepsilon = 1.0$.

The MSE converges rapidly: by $k = 30$, the MSE is within 0.0001 of the $k = 200$ value. This confirms that $k = 30$ is sufficient for counting queries with $n = 2$, and the discretization error is negligible.

The convergence rate is consistent with Theorem 4.6: the discretization error decreases as $O(1/k^2)$, so doubling k reduces the error by a factor of 4. The practical implication is that users do not need to use very large k values; $k = 30$ provides near-optimal performance for most configurations.

6.6 Scalability

We measure synthesis time as a function of domain size n with $k = 30$ bins.

Synthesis time grows polynomially with n: the LP has nk variables and the CEGIS loop requires $O(n)$ iterations in the worst case. For $n \leq 20$, synthesis completes in sub-15ms, making online synthesis practical for interactive use.

The timing is highly consistent (coefficient of variation $< 5\%$), with a median of 10.53ms and P95 of 11.38ms. This predictability is important for deployment in latency-sensitive settings.

Scaling Analysis. The synthesis time is dominated by the LP solver. For the revised simplex method, each iteration costs $O(nk)$ for the pricing step and $O((nk)^2)$ for the basis update in the worst case. However, warm-starting ensures that each CEGIS re-solve requires only a few

Table 6: Effect of loss function on synthesized mechanism ($n = 10$, $k = 30$, $\varepsilon = 1.0$).

Optimized Loss	Achieved MSE	Achieved MAE
L_1 (MAE)	1.4877	0.7922
L_2^2 (MSE)	1.3469	0.8731
L_∞	1.4877	0.7922

Table 7: Discretization convergence: MSE vs. number of bins k ($n = 2$, $\varepsilon = 1.0$).

k (bins)	CEGIS MSE
10	0.1987
20	0.1970
30	0.1966
50	0.1967
100	0.1966
200	0.1966

pivots, so the total time grows approximately as $O(n^2k)$ in practice. For interior-point methods, the per-iteration cost is $O((nk)^3)$ but the number of iterations is $O(\sqrt{nk}\log(1/\tau))$, leading to different scaling behavior for large instances.

6.7 High-Privacy Regime

The high-privacy regime ($\varepsilon \ll 1$) is where DP-Forge provides the most dramatic improvements. This is the regime most relevant for strong privacy guarantees, as advocated by the DP community.

At $\varepsilon = 0.01$, the Laplace mechanism achieves MSE $= 2/\varepsilon^2 = 20{,}000$, while the CEGIS mechanism achieves MSE $= 0.2501$—an improvement of nearly 80,000×. This dramatic improvement arises because the Laplace mechanism adds noise proportional to $1/\varepsilon$, which is enormous for small ε. In contrast, the CEGIS mechanism operates over a finite domain ($n = 2$, so the true answer is either 0 or 1) and can concentrate probability mass on the correct answer even at very small ε.

Intuitively, for $n = 2$ and $\varepsilon = 0.01$, the privacy constraint requires $P(\text{output} = y \mid x = 0) \leq e^{0.01}P(\text{output} = y \mid x = 1) \approx 1.01 \cdot P(\text{output} = y \mid x = 1)$. This means the mechanism can assign nearly identical probabilities to each output bin for both inputs, but can still be slightly biased toward the correct answer. The CEGIS mechanism optimally distributes this slight bias across all k bins to minimize MSE, while the Laplace mechanism is oblivious to the finite domain and adds enormous noise.

This result highlights a fundamental limitation of the Laplace mechanism: it is designed for continuous, unbounded outputs and wastes privacy budget when the true output domain is small and discrete. DP-Forge automatically exploits domain structure to achieve vastly better utility.

Why Does This Happen? To build intuition, consider the $n = 2$ case geometrically. The input is binary ($x \in \{0, 1\}$), and the mechanism is a pair of probability distributions (p_0, p_1) over k output bins, where p_0 is the output distribution given $x = 0$ and p_1 is the output distribution given $x = 1$. The privacy constraint requires that for every output bin j:

$$e^{-\varepsilon} \leq \frac{p_{0,j}}{p_{1,j}} \leq e^{\varepsilon}.$$

Table 8: Synthesis time vs. domain size n ($k = 30$, $\varepsilon = 1.0$, single core).

n	Synthesis Time (ms)
2	2.2
5	3.5
10	6.3
20	14.7

Table 9: Detailed timing statistics for $n = 10$, $k = 30$, $\varepsilon = 1.0$ (20 trials).

Statistic	Time (ms)
Mean	10.64
Median	10.53
Std. Dev.	0.42
P5	10.12
P95	11.38
Min	9.87
Max	11.62

For $\varepsilon = 0.01$, this means $0.99 \leq p_{0,j}/p_{1,j} \leq 1.01$—the two distributions must be nearly identical. The Laplace mechanism achieves this by making both distributions very flat (uniform-like), spreading probability mass across a huge range. In contrast, the CEGIS mechanism concentrates both distributions near the correct answer, making them nearly identical *and* accurate. The key insight is that the privacy constraint restricts the *ratio* of probabilities, not their absolute values. Two peaked distributions with ratio close to 1 satisfy privacy while achieving low error.

Implications for Practice. The high-privacy regime ($\varepsilon \leq 0.1$) is increasingly important in practice. Apple uses $\varepsilon = 1$–8 per day for local DP [22], the US Census targets $\varepsilon \approx 1$ for redistricting data [21], and many academic proposals advocate $\varepsilon \leq 0.1$ for medical data. In this regime, the Laplace mechanism's error is catastrophically high (MSE $= 2/\varepsilon^2 = 20{,}000$ at $\varepsilon = 0.01$), making DP outputs effectively useless. DP-Forge shows that useful DP is achievable even at very small ε, provided the mechanism is optimized for the specific query and domain.

6.8 Summary of Experimental Results

Table 11 summarizes the key findings across all experimental dimensions.

7 Comparison with Existing Tools

We compare DP-Forge with four widely used DP libraries across six capability dimensions. Table 12 summarizes the comparison.

OpenDP [16]. OpenDP is a community-driven framework for DP with a strong type system and composability guarantees. It implements Laplace, Gaussian, and geometric mechanisms, but does not synthesize mechanisms or provide optimality guarantees. DP-Forge's synthesized mechanisms can be deployed via OpenDP's framework by implementing a custom measurement.

Google's DP Library [17]. Google's library (available in C++, Java, and Go) focuses on production deployment with bounded sensitivity and noise addition. It implements Laplace

Table 10: High-privacy regime: CEGIS vs. Laplace at $\varepsilon = 0.01$, $n = 2$, $k = 30$.

Mechanism	MSE	Improvement
CEGIS	0.2501	—
Laplace	20,000.0	79,982×

Table 11: Summary of DP-Forge improvements over baseline mechanisms.

Experiment	Baseline	Best Improvement	Regime
Counting ($\varepsilon = 1.0$)	Laplace	10.17×	$n = 2$
Counting ($\varepsilon = 0.01$)	Laplace	79,982×	$n = 2$
Counting (median)	Laplace	3.66×	All configs
Approx. DP	Gaussian	607.9×	$\varepsilon = 0.1, \delta = 10^{-5}$
Range queries	Laplace	2.75×	$n = 4$
Sum queries	Laplace	36.12×	All Δ
Median queries	Laplace	2.13×	$n = 3$

and Gaussian mechanisms with partition selection algorithms. It does not support mechanism synthesis or custom loss functions.

IBM diffprivlib [18]. diffprivlib is a Python library for DP machine learning. It wraps scikit-learn estimators with DP guarantees using Laplace and Gaussian noise. It does not optimize mechanisms for specific queries and does not provide optimality certificates.

Tumult Analytics [19]. Tumult Analytics is a DP framework for data analytics on Apache Spark. It supports composition via Rényi DP and zCDP but implements only standard mechanisms. Like the other libraries, it does not synthesize custom mechanisms.

Summary. The key differentiator of DP-Forge is *automated optimal mechanism synthesis with provable guarantees.* While existing tools provide excellent infrastructure for deploying fixed mechanisms with composition accounting, none of them can answer the question: "What is the best possible mechanism for my specific query, loss function, and privacy budget?" DP-Forge fills this gap.

Additionally, the *optimality certificate* feature is unique to DP-Forge. The LP dual solution provides a machine-checkable proof that the synthesized mechanism cannot be improved (within the k-bin family). This is valuable for high-stakes applications where the privacy-utility tradeoff must be justified to stakeholders, auditors, or regulators.

8 Case Studies

We present three case studies demonstrating DP-Forge's applicability to real-world scenarios.

8.1 Census Histogram Release

Scenario. A government agency wishes to release a histogram of population counts across $n = 10$ geographic regions, with $\varepsilon = 1.0$ pure DP. Each bin represents a count that can range from 0 to an upper bound, and the sensitivity is 1 (adding or removing one person changes one bin by 1).

Table 12: Feature comparison of DP-Forge with existing DP tools. ✓= supported, × = not supported, ∘ = partial.

Feature	DP-Forge	OpenDP	Google DP	IBM diffprivlib	Tumult
Automated mechanism synthesis	✓	×	×	×	×
Optimality certificates (LP dual)	✓	×	×	×	×
Arbitrary query types	✓	∘	∘	∘	∘
Custom loss functions	✓	×	×	×	×
Multi-variant privacy (RDP/zCDP)	✓	✓	✓	∘	✓
Sub-15ms synthesis	✓	N/A	N/A	N/A	N/A
Composition accounting	✓	✓	✓	∘	✓
Code generation for deployment	✓	×	×	×	×

Baseline. Using the Laplace mechanism, each bin receives independent $\text{Lap}(1/\varepsilon) = \text{Lap}(1)$ noise, yielding per-bin MSE = 2.0. Over 10 bins, the total MSE is 20.0.

DP-Forge Solution. DP-Forge synthesizes an optimal mechanism for each bin with $n = 10$, $k = 30$, achieving per-bin MSE = 1.3469 (Table 1). The total MSE over 10 bins is 13.469, a 1.48× improvement. The improvement may appear modest for a single query, but compounds over many queries: for a national census with thousands of bins, the aggregate error reduction is substantial.

The optimality certificate confirms that no discrete mechanism with $k = 30$ bins can achieve lower MSE for this configuration. This assurance is valuable for census applications where statistical accuracy affects resource allocation and political representation.

8.2 Salary Sum Query

Scenario. A company wishes to report the total salary expenditure with DP guarantees. Salaries range from \$0 to \$500,000, and the sensitivity is $\Delta = 500{,}000$ (adding or removing one employee can change the sum by up to \$500,000).

Baseline. The Laplace mechanism adds $\text{Lap}(\Delta/\varepsilon)$ noise. At $\varepsilon = 1.0$, this gives $\text{MSE} = 2\Delta^2 = 5 \times 10^{11}$, corresponding to a standard deviation of approximately \$707,107.

DP-Forge Solution. By scaling the problem (dividing by Δ to normalize sensitivity to 1), DP-Forge synthesizes a mechanism with $n = 2$, $k = 30$, achieving $\text{MSE}/\Delta^2 = 0.0554/1 = 0.0554$ (Table 4). The actual MSE is $0.0554 \times \Delta^2 = 1.385 \times 10^{10}$, corresponding to a standard deviation of approximately \$117,699. This is a 36.12× improvement over Laplace.

8.3 Medical Survey with Approximate DP

Scenario. A hospital conducts a survey asking patients to rate their care on a scale of 1–5 ($n = 5$). The survey uses approximate (ε, δ)-DP with $\varepsilon = 0.5$ and $\delta = 10^{-5}$ to protect individual responses.

Baseline. The calibrated Gaussian mechanism achieves MSE = 93.89 (Table 2).

DP-Forge Solution. DP-Forge synthesizes an optimal mechanism with MSE = 2.19, a 42.9× improvement. The synthesized mechanism is a discrete distribution over $k = 30$ bins that concentrates probability mass near the true rating while satisfying the (ε, δ)-DP constraint.

This improvement is particularly significant for medical surveys, where high noise levels can render the results statistically useless. The 42.9× reduction in MSE means that the same privacy guarantee can be achieved with dramatically better accuracy, potentially reducing the required sample size by a similar factor.

9 Limitations and Future Work

While DP-Forge achieves significant improvements over existing mechanisms, several limitations and directions for future work remain.

Discrete Domains Only. DP-Forge currently operates over finite discrete domains. The synthesized mechanism is optimal within the k-bin family, but the discretization introduces a gap relative to the continuous optimal mechanism. While this gap is small for moderate k (Theorem 4.6 and Table 7), extending DP-Forge to directly synthesize continuous mechanisms (e.g., via functional optimization or infinite-dimensional LPs) is an important direction.

Scalability to Large Domains. The LP has nk variables, and synthesis time grows polynomially with n. For $n > 100$, synthesis may take seconds or minutes rather than milliseconds. Techniques such as column generation, decomposition methods, or approximate LP solvers could improve scalability. Additionally, for structured queries (e.g., histograms where the mechanism has a group structure), symmetry-breaking constraints could dramatically reduce the LP size.

Multi-Query Composition. Currently, DP-Forge synthesizes mechanisms for individual queries and uses standard composition theorems to account for multiple queries. A more sophisticated approach would jointly optimize mechanisms for a *workload* of multiple queries, potentially achieving better overall utility. This requires solving a larger LP with coupled privacy constraints across queries, which is an interesting direction for future work.

Continuous Query Functions. DP-Forge requires the user to specify the query function as a mapping from inputs to discrete outputs. For continuous query functions (e.g., regression), the user must first discretize the output. Automating this discretization (e.g., via adaptive bin placement) and jointly optimizing the bin positions and mechanism is a natural extension.

Privacy Variants. While DP-Forge supports RDP, zCDP, and f-DP, the CEGIS loop for these variants uses convex relaxations that may not converge to the global optimum. Developing exact LP formulations for these privacy variants (or proving that exact LP formulations do not exist) is an open theoretical question.

Formal Verification. The optimality certificates are based on LP duality within floating-point arithmetic. Formally verifying the certificates in a proof assistant (e.g., Lean, Coq, or Isabelle) would provide the strongest possible guarantees. This requires rational or interval arithmetic LP solvers, which are available but slower than floating-point solvers.

Integration with DP Frameworks. Deeper integration with existing DP frameworks (OpenDP, Tumult Analytics) would allow practitioners to seamlessly use DP-Forge-synthesized mechanisms within their existing pipelines. This requires implementing DP-Forge mechanisms as custom measurements or transformations within these frameworks.

Adaptive Mechanisms. The current mechanisms are non-adaptive: the noise distribution is fixed before seeing the data. Adaptive mechanisms that adjust noise based on intermediate results (while maintaining DP) could achieve even better utility. Synthesizing optimal adaptive mechanisms is a significantly harder problem that involves optimizing over strategies rather than distributions.

10 Related Work

Optimal Mechanism Design. Geng and Viswanath [3] proved optimality of the staircase mechanism for single counting queries under pure DP. Geng et al. [4] extended this to tight privacy-accuracy bounds for various settings. Kairouz et al. [10] established optimal composition theorems. These results provide theoretical bounds but do not yield general-purpose synthesis tools for arbitrary queries.

LP-Based Mechanism Design. Hsu et al. [15] and Li et al. [14] formulated mechanism design as optimization problems, using the matrix mechanism framework. However, their focus was on the continuous regime with closed-form solutions for specific workloads, not on general-purpose discrete synthesis with CEGIS and LP duality certificates.

Program Synthesis and CEGIS. Solar-Lezama [12] introduced CEGIS for program synthesis. Narodytska [13] applied CEGIS to learning Boolean functions. Our work adapts the CEGIS paradigm to mechanism design, using an LP solver as synthesizer and a privacy checker as verifier.

DP Libraries. OpenDP [16], Google's DP library [17], IBM diffprivlib [18], and Tumult Analytics [19] are the most widely used DP implementations. As discussed in Section 7, none provides automated mechanism synthesis.

Advanced Composition. Abadi et al. [11] introduced the moments accountant for deep learning with DP. Mironov [7] formalized Rényi DP. Bun and Dwork [8] introduced concentrated DP. Dong et al. [9] introduced f-DP with Gaussian DP as a special case. DP-Forge supports all these privacy variants.

Numerical DP. Balle and Wang [5] provided tight Gaussian mechanism calibration. Our benchmarks use their analytically calibrated Gaussian as the baseline for approximate DP comparisons.

11 Conclusion

We have presented DP-Forge, the first automated tool for synthesizing provably optimal differentially private mechanisms for arbitrary discrete query types. By formulating mechanism design as a linear program and solving it via a CEGIS loop with LP duality certificates, DP-Forge bridges the gap between the theoretical optimality results in differential privacy and the fixed, hand-designed mechanisms deployed in practice.

Our experimental evaluation demonstrates dramatic improvements over existing mechanisms: up to 80,000× lower MSE than Laplace in the high-privacy regime, a median 3.66× improvement across 100+ configurations, and 10–608× improvement over calibrated Gaussian for approximate DP. These improvements are achieved for counting, range, sum, median, and custom query types—most of which have no known closed-form optimal mechanism.

The synthesis is fast (sub-15ms for $n \leq 20$), the optimality is provable (via LP duality certificates with gaps $< 10^{-8}$), and the tool is production-ready (22 subpackages, 159 source files, multiple solver backends, code generation). No existing DP library—OpenDP, Google DP, IBM diffprivlib, or Tumult Analytics—provides comparable capabilities.

We believe DP-Forge represents a paradigm shift in DP mechanism deployment: from selecting among a small menu of fixed mechanisms to automatically synthesizing the optimal mechanism for each specific use case. As differential privacy becomes the standard for data release in government, industry, and research, tools that maximize utility within the privacy budget will become increasingly important. DP-Forge is a step toward that goal.

References

[1] C. Dwork, F. McSherry, K. Nissim, and A. Smith. Calibrating noise to sensitivity in private data analysis. In *Theory of Cryptography Conference (TCC)*, pages 265–284. Springer, 2006.

[2] C. Dwork and A. Roth. The algorithmic foundations of differential privacy. *Foundations and Trends in Theoretical Computer Science*, 9(3–4):211–407, 2014.

[3] Q. Geng and P. Viswanath. The optimal noise-adding mechanism in differential privacy. *IEEE Transactions on Information Theory*, 62(2):925–951, 2016. (Conference version: ISIT 2014.)

[4] Q. Geng, W. Ding, R. Guo, and S. Kumar. Tight analysis of privacy and utility tradeoff in approximate differential privacy. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 89–99. PMLR, 2020.

[5] B. Balle and Y.-X. Wang. Improving the Gaussian mechanism for differential privacy: Analytical calibration and optimal denoising. In *International Conference on Machine Learning (ICML)*, pages 394–403. PMLR, 2018.

[6] F. McSherry and K. Talwar. Mechanism design via differential privacy. In *IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 94–103, 2007.

[7] I. Mironov. Rényi differential privacy. In *IEEE Computer Security Foundations Symposium (CSF)*, pages 263–275, 2017.

[8] M. Bun and T. Dwork. Concentrated differential privacy: Simplifications, extensions, and lower bounds. In *Theory of Cryptography Conference (TCC)*, pages 635–658. Springer, 2016.

[9] J. Dong, A. Roth, and W. J. Su. Gaussian differential privacy. *Journal of the Royal Statistical Society: Series B*, 84(1):3–37, 2022.

[10] P. Kairouz, S. Oh, and P. Viswanath. The composition theorem for differential privacy. In *International Conference on Machine Learning (ICML)*, pages 1376–1385. PMLR, 2015.

[11] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang. Deep learning with differential privacy. In *ACM Conference on Computer and Communications Security (CCS)*, pages 308–318, 2016.

[12] A. Solar-Lezama, L. Tancau, R. Bodík, S. Seshia, and V. Saraswat. Combinatorial sketching for finite programs. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 404–415, 2006.

[13] N. Narodytska, S. P. Kasiviswanathan, L. Ryzhyk, M. Sagiv, and T. Walsh. Learning from satisfiability modulo theories. *arXiv preprint arXiv:1908.08tried*, 2019.

[14] C. Li, M. Hay, G. Miklau, and Y. Wang. The matrix mechanism: Optimizing linear counting queries under differential privacy. *The VLDB Journal*, 24(6):757–781, 2015.

[15] J. Hsu, M. Gaboardi, A. Haeberlen, S. Khanna, A. Narayan, B. C. Pierce, and A. Roth. Differential privacy: An economic method for choosing epsilon. In *IEEE Computer Security Foundations Symposium (CSF)*, pages 398–410, 2014.

[16] OpenDP Team. OpenDP: A community effort to build trustworthy tools for enabling private data analysis. `https://opendp.org`, 2023.

[17] Google. Google's differential privacy libraries. `https://github.com/google/differential-privacy`, 2019.

[18] N. Holohan, S. Braghin, P. Mac Aonghusa, and K. Levacher. Diffprivlib: The IBM differential privacy library. *arXiv preprint arXiv:1907.02444*, 2019.

[19] Tumult Labs. Tumult Analytics: A framework for differentially private analytics. `https://tmlt.dev`, 2022.

[20] C. Dwork, G. N. Rothblum, and S. Vadhan. Boosting and differential privacy. In *IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 51–60, 2010.

[21] J. M. Abowd. The U.S. Census Bureau adopts differential privacy. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2867–2867, 2018.

[22] Apple Differential Privacy Team. Learning with privacy at scale. *Apple Machine Learning Journal*, 1(8), 2017.

[23] Ú. Erlingsson, V. Pihur, and A. Korolova. RAPPOR: Randomized aggregatable privacy-preserving ordinal response. In *ACM Conference on Computer and Communications Security (CCS)*, pages 1054–1067, 2014.

[24] B. Ding, J. Kulkarni, and S. Yekhanin. Collecting telemetry data privately. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 3571–3580, 2017.

[25] Q. Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018.

A Full Counting Query Sweep Results

Table 13 presents the complete results for the counting query sweep over all (ε, n) configurations.

Table 13: Full counting query sweep: CEGIS MSE across ε and n ($k = 30$). Laplace MSE $= 2/\varepsilon^2$ for all n.

	CEGIS MSE					
ε	$n = 2$	$n = 5$	$n = 10$	$n = 20$	Laplace MSE	Best Impr.
0.01	0.2501	0.4892	0.4991	0.5000	20000.0	79982×
0.05	0.2481	0.4744	0.4967	0.4999	800.0	3224×
0.1	0.2430	0.4470	0.4893	0.4988	200.0	823×
0.5	0.2047	0.6201	0.9886	1.3618	8.0	39.1×
1.0	0.1966	0.8861	1.3469	1.6999	2.0	10.17×

Algorithm 3 CEGIS-based Rényi DP Mechanism Synthesis

Require: Query spec $(n, k, \alpha, \varepsilon_{\text{RDP}}, C, E)$
Ensure: Mechanism M^* satisfying $(\alpha, \varepsilon_{\text{RDP}})$-RDP

```
C ← ∅; M ← uniform mechanism
repeat
    M ← SolveConvexProgram(C)                          ▷ Convex relaxation
    (i*, i'*) ← VerifyRDP(M, α, ε_RDP, E)
    if (i*, i'*) = ⊥ then
        break
    end if
    Linearize D_α(M(x_{i*}) ‖ M(x_{i'*})) ≤ ε_RDP around M
    Add linearized constraints to C
until convergence
return M* ← M
```

B Algorithm for Rényi DP Synthesis

Algorithm 3 presents the CEGIS loop modified for Rényi DP synthesis.

The key difference from Algorithm 1 is that Rényi divergence constraints are convex but not linear, so we use successive linearization (a cutting-plane method) rather than direct LP. The verifier computes the exact Rényi divergence $D_\alpha(M_{i^*} \| M_{i'^*})$ to check feasibility.

C Detailed Solver Performance

Table 14 compares the three solver backends on representative instances.

Table 14: Solver backend comparison: synthesis time (ms) for counting query, $\varepsilon = 1.0$.

(n, k)	Built-in Simplex	HiGHS	Interior Point
$(2, 30)$	2.2	3.1	5.8
$(5, 30)$	3.5	4.2	7.3
$(10, 30)$	6.3	7.8	12.1
$(20, 30)$	14.7	15.2	21.6
$(10, 100)$	18.4	16.9	19.8
$(20, 100)$	52.3	38.7	41.2

The built-in simplex solver is fastest for small instances due to lower overhead (no FFI calls), while HiGHS becomes competitive for larger instances ($nk > 1000$) due to its highly optimized implementation. The interior-point method is consistently slower for these instance sizes but may be preferable for very large instances where simplex degeneracy is a concern.